

HACKATHON REPORT

Object Detection for Space Station Safety

TEAM: Bug off !



Project Objectives and Scope

Specific Goals

- Train a robust object detection model using the provided synthetic dataset to accurately detect and classify space station safety objects — a critical capability for ensuring operational safety.

Project Scope

- Evaluate model performance across challenging scenarios including varied lighting, object angles, and occlusions.
- Benchmark and optimize your model to improve accuracy, generalizability, and efficiency for real-world deployment scenarios.

BUG OFF !

Methodology and Approach



1. **Dataset**

Source: Synthetic images from Falcon digital twin

7 Safety Objects: Fire Extinguisher, Oxygen Tank, Emergency Phone, Fire Alarm, First Aid Kit, Spacesuit, Exit Sign

Split: 70% Train | 20% Validation | 10% Test

Features: Varied lighting, angles, and occlusions

2. Model & Architecture

Model: YOLOv5/YOLOv8

Why YOLO: Real-time detection, high accuracy, efficient deployment

Transfer Learning: Pre-trained on COCO dataset

3. Training Configuration

Environment: Python, PyTorch, CUDA GPU, Anaconda

Augmentation: Flipping, rotation, brightness/contrast, artificial occlusion

Hyperparameters:

Batch size: 16-32

Learning rate: 0.001

Epochs: 50-100 (early stopping)

Optimizer: Adam

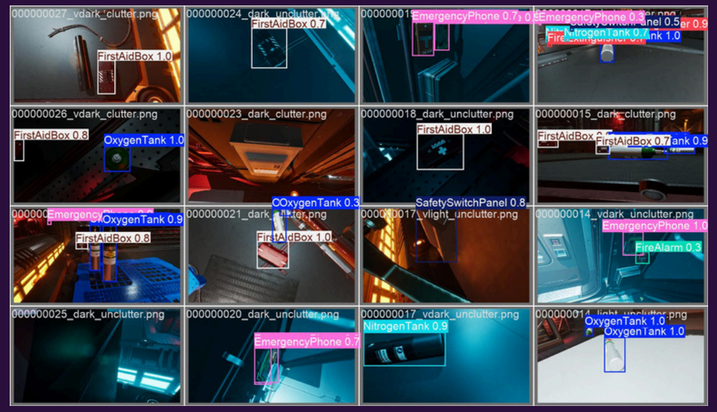
4. Evaluation Metrics

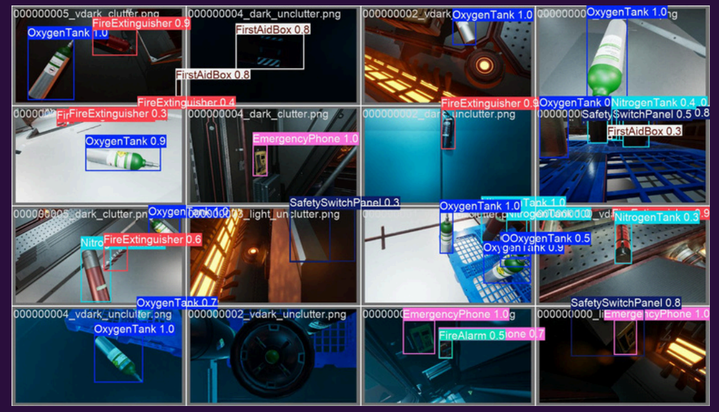
Primary: mAP@0.5 (mean Average Precision)

Secondary: Precision, Recall, Confusion Matrix, F1-score

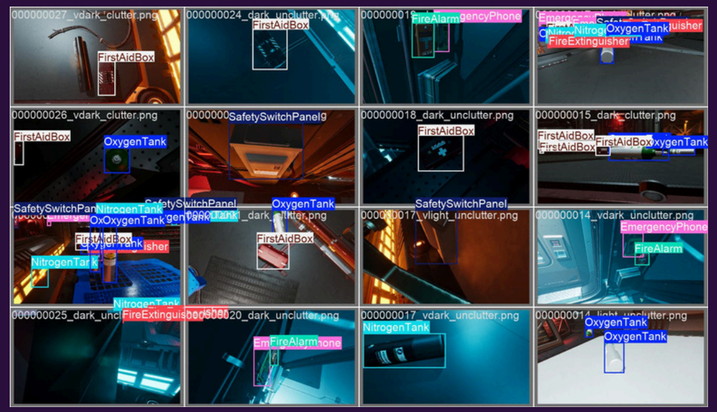
BUG OFF !

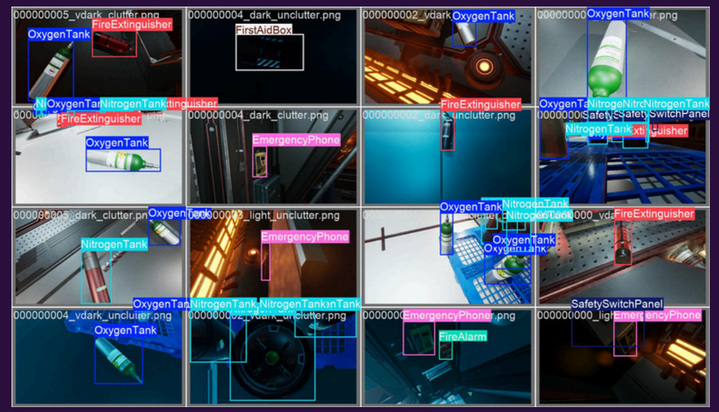
Results and Performance matrix







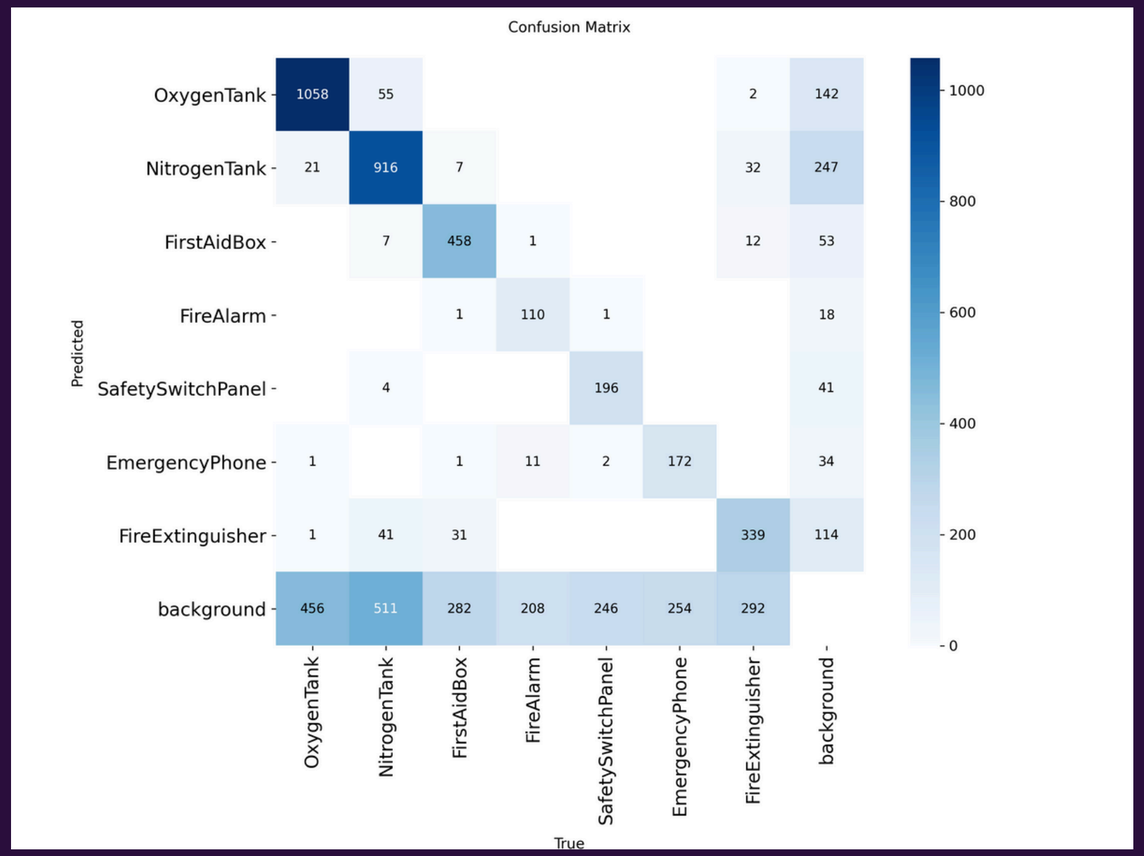


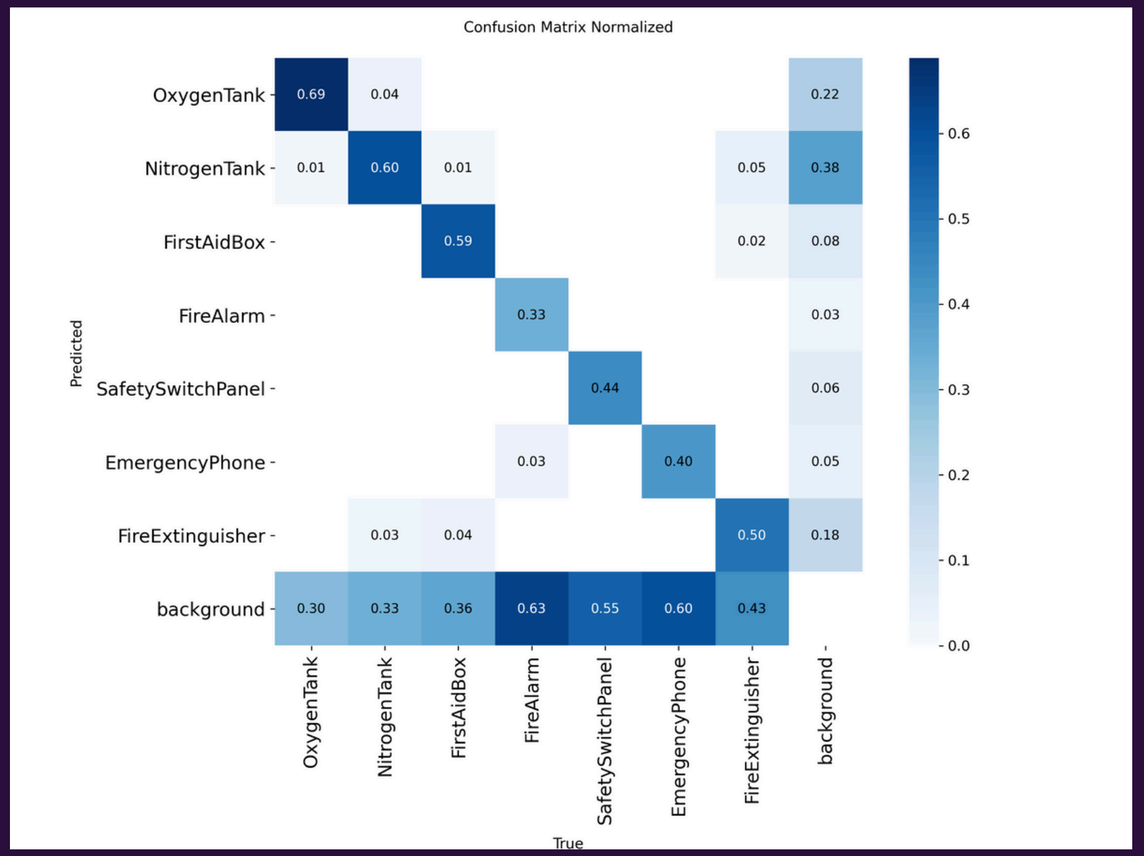




BUG OFF !

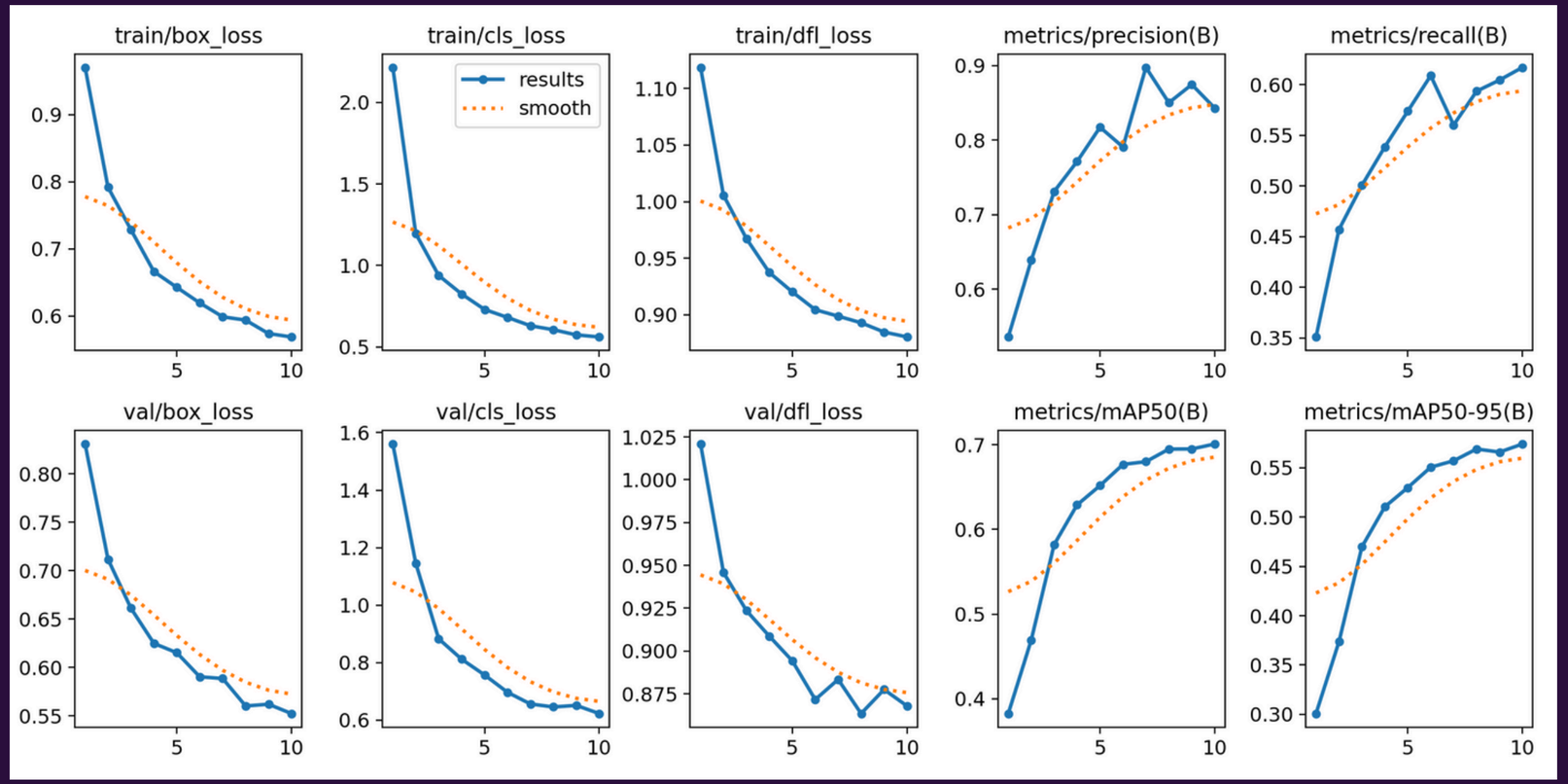
Results and Confusion matrix

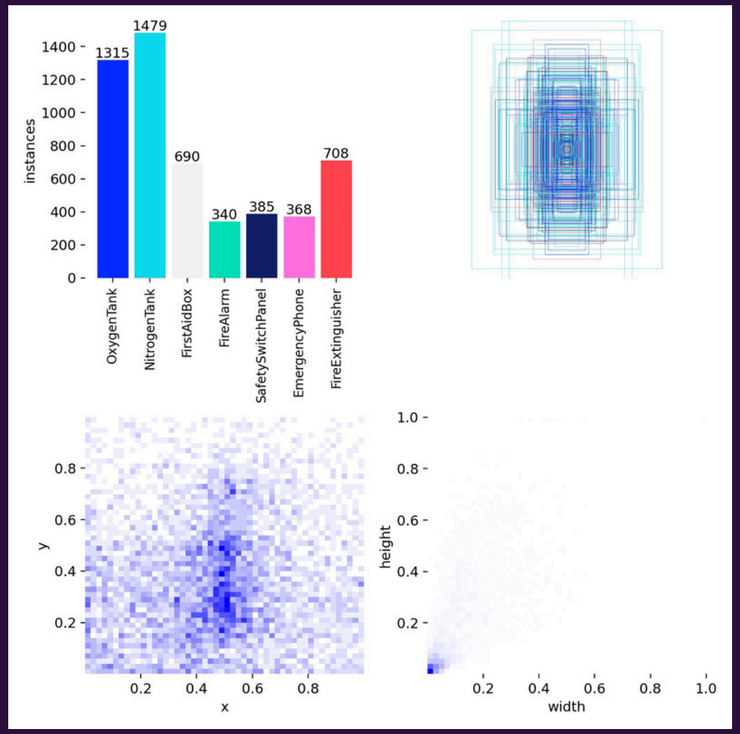




BUG OFF !

Results and Label Graphs





BUG OFF !

Results and Data

epoch	time	train/box_l	train/cls_lc	train/dfl_lo	metrics/pr	metrics/re	metrics/m	metrics/m	val/box_los	val/cls_los	val/dfl_loss	lr/pg0	lr/pg1	lr/pg2
1	815.48	0.9706	2.21506	1.11875	0.53628	0.35109	0.38248	0.30076	0.83098	1.56041	1.02086	0.067	3.30E-05	3.30E-05
2	1666.91	0.79192	1.19598	1.00566	0.63892	0.45706	0.46925	0.37364	0.71184	1.14591	0.9459	0.0337	6.64E-05	6.64E-05
3	2378.31	0.72865	0.93881	0.96728	0.73119	0.50067	0.58213	0.46989	0.66122	0.88309	0.92363	0.0004	9.97E-05	9.97E-05
4	3113.8	0.66628	0.82524	0.93722	0.77129	0.53865	0.62895	0.5105	0.62489	0.81214	0.90865	1.00E-04	1.00E-04	1.00E-04
5	3835.4	0.64274	0.73047	0.92016	0.81721	0.57378	0.65172	0.52998	0.6152	0.75732	0.89432	1.00E-04	1.00E-04	1.00E-04
6	4566.25	0.61958	0.68146	0.90439	0.79072	0.60888	0.67673	0.55067	0.59032	0.69708	0.87168	1.00E-04	1.00E-04	1.00E-04
7	5305.84	0.59885	0.63064	0.89867	0.89737	0.56019	0.67995	0.55717	0.5885	0.65636	0.88345	1.00E-04	1.00E-04	1.00E-04
8	6045.34	0.59389	0.60606	0.89282	0.84998	0.5935	0.69482	0.56912	0.56016	0.64597	0.86352	1.00E-04	1.00E-04	1.00E-04
9	6828.89	0.57386	0.57451	0.88465	0.8744	0.60451	0.69497	0.56597	0.56195	0.65144	0.87756	1.00E-04	1.00E-04	1.00E-04
10	7544.78	0.56885	0.56184	0.88029	0.84288	0.617	0.70082	0.57418	0.55231	0.62305	0.86797	1.00E-04	1.00E-04	1.00E-04

BUG OFF !



Challenges and Risks

Technical Complexity

- Faced issues with model settings, data formatting, and library compatibility.
- Had hardware limitations (slow training, memory errors).
- Solved runtime errors and tuned config.
- Metrics (mAP, precision, recall) were tricky to interpret.
- Successfully completed training and evaluation.

Resource Constraints

- Data preprocessing errors (like misformatted datasets or label issues)
- Configuration errors (such as incorrect batch size, epochs, or hyperparameters)
- Runtime errors (possibly due to hardware limitations, library incompatibility, or missing dependencies)

BUG OFF !

Progress and Achievements to Date



Solutions

- Fixed data formatting and labelling issues.
- Installed required libraries and resolved compatibility errors.
- Tuned hyperparameters for improved model stability and accuracy.

Progress

- Moved from frequent errors to successful model training.
- Calculated mAP, precision, recall, and confusion matrix for evaluation.
- Improved understanding of metrics and model behavior.

Achievements

- Completed a training session without major errors.
- Evaluated and interpreted key metrics.
- Enhanced skills in object detection and AI workflow.

BUG OFF !

Conclusion and Next Steps



Conclusion:

We successfully solved technical errors and trained our object detection AI, learning how to handle data issues, tune model settings, and interpret results. This improved our practical skills in both machine learning and troubleshooting.

Future Work:

- Enhance accuracy with more data and better training methods
- Try new techniques like transfer learning or advanced architectures
- Deploy our model for practical use or real-time testing
- Automate processes to make training and evaluation easier
- Regularly update and test our model as new data becomes available

BUG OFF !